

Project Report — Customer Support Agent: Complete Support Automation

1. Introduction

Customer support teams handle a high volume of repetitive queries alongside sensitive, high-risk cases that require human judgment. This project implements an AI-powered Customer Support Agent using `n8n`, an AI Agent backed by a `Groq`-hosted LLM, and a rule-based escalation mechanism, so that safe queries are answered automatically while risky ones are routed to a human.

2. Problem Statement

Manually triaging and responding to every customer message is time-consuming and inconsistent across agents. There is a need for a system that can understand a customer's request, classify it accurately, and decide — without human involvement for low-risk cases — whether it can be safely resolved automatically or must be handed off to a human support representative.

3. Objectives

- Automatically classify customer messages by category, urgency, and sentiment.
- Generate a professional, empathetic, and policy-safe response.
- Decide between `auto_resolve` and `escalate` based on defined risk rules.
- Maintain conversational context across a session.
- Produce structured, machine-readable output for downstream ticket routing.

4. Proposed Solution

An `n8n` workflow triggered by a chat message. The message is sent to an AI Agent node backed by a `Groq` LLM (`openai/gpt-oss-120b`), which uses a fixed system prompt, session memory, and a structured output parser to return a classified, structured JSON result. An IF node routes the result to one of two "Edit Fields" branches (escalation or auto-resolution), both of which converge on a single Chat node that sends the final customer-facing response.

5. System Architecture

...

Customer → Chat Trigger → AI Agent (Groq LLM + Memory + Output Parser)

→ IF (action == escalate?)

■■■ TRUE → Edit Fields (ticket_status=Escalated, ticket_id)

■■■ FALSE → Edit Fields1 (ticket_status=Auto-Resolved)

→ Chat (Send Message)

...

Full diagram and node explanations are provided in `ARCHITECTURE.md`.

6. Workflow Description

1. The customer sends a message through the n8n Chat Trigger.
2. The AI Agent processes the message using the system prompt, prior conversation memory, and the Groq LLM.
3. The Structured Output Parser enforces a fixed JSON schema on the AI Agent's response.
4. An IF node checks the `action` field; `escalate` routes to the escalation branch, anything else routes to the auto-resolution branch.
5. Each branch (`Edit Fields` / `Edit Fields1`) builds a structured record (ticket status, category, urgency, sentiment, response, reason, and — for escalations — a generated ticket ID).
6. The `Chat` node sends `customer\_response` back to the user.

7. Technologies Used

- **n8n** (workflow automation engine)
- **Groq API** — LLM inference (openai/gpt-oss-120b)
- **n8n LangChain integration**: Chat Trigger, AI Agent, Chat Model (Groq), Memory Buffer Window, Structured Output Parser, Chat (Send Message)
- **n8n core nodes**: IF, Set

8. AI Agent Design

The AI Agent is configured with `hasOutputParser: true` and a detailed system message defining its role, classification taxonomy, escalation rules, safety rules, and required JSON output shape. It is connected to three sub-inputs: the Groq Chat Model (language model), Simple Memory (conversation memory), and the Structured Output Parser (output format enforcement).

9. Prompt Engineering

The system prompt is structured into clearly separated sections: role definition, category/urgency/sentiment taxonomies, explicit escalation triggers, auto-resolve criteria, anti-hallucination rules, the required JSON output schema, and an embedded FAQ knowledge base. This separation makes the model's behavior predictable and auditable. Full prompt text is in `AI\_PROMPT\_LIBRARY.md`.

10. Knowledge Base / FAQ

A concise FAQ is embedded directly in the system prompt, covering password reset, order tracking, profile updates, login problems, billing problems, and refund/cancellation guidance. The agent is explicitly instructed to treat this as general guidance only and never invent company-specific policies, prices, or promises beyond it.

11. Conversation Memory

The `Simple Memory` node (`memoryBufferWindow`, context window = 10) retains recent conversation turns, keyed to the Chat Trigger's session ID, allowing the agent to reference earlier messages in the same session for continuity.

12. Classification System

Every message is classified into one of eight categories: Account, Billing, How-to, Troubleshooting, Order/Delivery, Refund/Cancellation, Complaint, Other.

13. Urgency Detection

Messages are scored as Low, Medium, High, or Critical based on the nature and impact of the issue described.

14. Sentiment Detection

Messages are scored as Positive, Neutral, Negative, or Very Negative, which — combined with category — feeds into the escalation decision.

15. Auto-Resolution

Applied when the issue can be safely answered using general support guidance without needing a human decision (e.g., password reset, general how-to questions). The ``Edit Fields1`` node marks these as ``ticket_status = "Auto-Resolved"``.

16. Human Escalation

Applied for refunds/cancellations involving money, financial disputes, duplicate/incorrect charges, payment problems, account security concerns, legal concerns, serious complaints, and very negative sentiment. The ``Edit Fields`` node marks these as ``ticket_status = "Escalated"`` and generates a ``ticket_id``.

17. Structured Output

The Structured Output Parser enforces exactly six fields on every AI Agent response — ``category``, ``urgency``, ``sentiment``, ``action``, ``response``, ``reason`` — nested under ``$json.output``, ensuring reliable downstream processing without free-text parsing.

18. Ticket Routing

Routing is handled entirely by the ``If`` node checking ``$json.action == "escalate"``, directing execution to the appropriate ``Edit Fields`` branch, which prepares the ticket record fields consumed by the final response node (and, in a production system, could be forwarded to an external ticketing tool).

19. Testing

Two end-to-end tests were executed on the live, published workflow:

- **Auto-resolve test:** password-reset query, correctly classified and auto-resolved with FAQ-based guidance.
- **Escalation test:** duplicate-charge/refund query, correctly classified as Billing/High/Negative and escalated with a generated ticket ID.

Full results are documented in ``TEST_CASES.md``, with screenshot evidence in ``screenshots/``.

20. Results

Both executed tests produced correct classification, correct routing decisions, and policy-safe responses (no invented refund amounts, policies, or promises), confirming the workflow meets its core design goals.

21. Security and Safety Considerations

- The system prompt explicitly forbids inventing company policies, prices, refund rules, delivery promises, or account information.
- Sensitive actions (refunds, payment disputes, security issues, legal concerns) are never auto-resolved — they are always escalated to a human.
- No API keys or credentials are stored in the workflow JSON or exposed in this documentation.

22. Limitations

- Relies on a single LLM provider (Groq) with no fallback.
- Escalation only produces internal ticket fields; there is no live integration with an external ticketing/helpdesk system.
- FAQ knowledge is static and embedded in the prompt rather than a queryable external knowledge base.
- No explicit customer identity verification step.

23. Future Scope

- Integrate with a real ticketing system via API (e.g., Zendesk/Freshdesk).
- Move the FAQ to a vector database for retrieval-augmented generation (RAG) as the knowledge base grows.
- Add customer-tier-aware routing.
- Add a broader automated test suite.

24. Conclusion

This project demonstrates a working AI-driven customer support automation pipeline built entirely in n8n, combining an LLM-based AI Agent, structured output enforcement, conversation memory, and rule-based escalation logic. It satisfies the assignment's core requirement of automating safe, repetitive support work while keeping high-risk cases under human supervision.